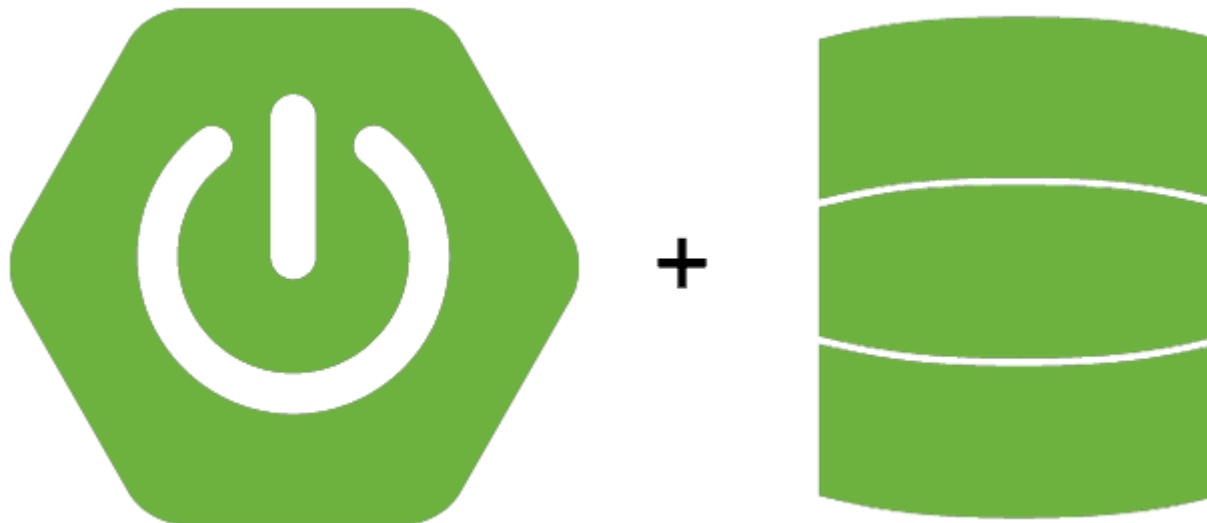


SPRING DATA JPA – JPARepository



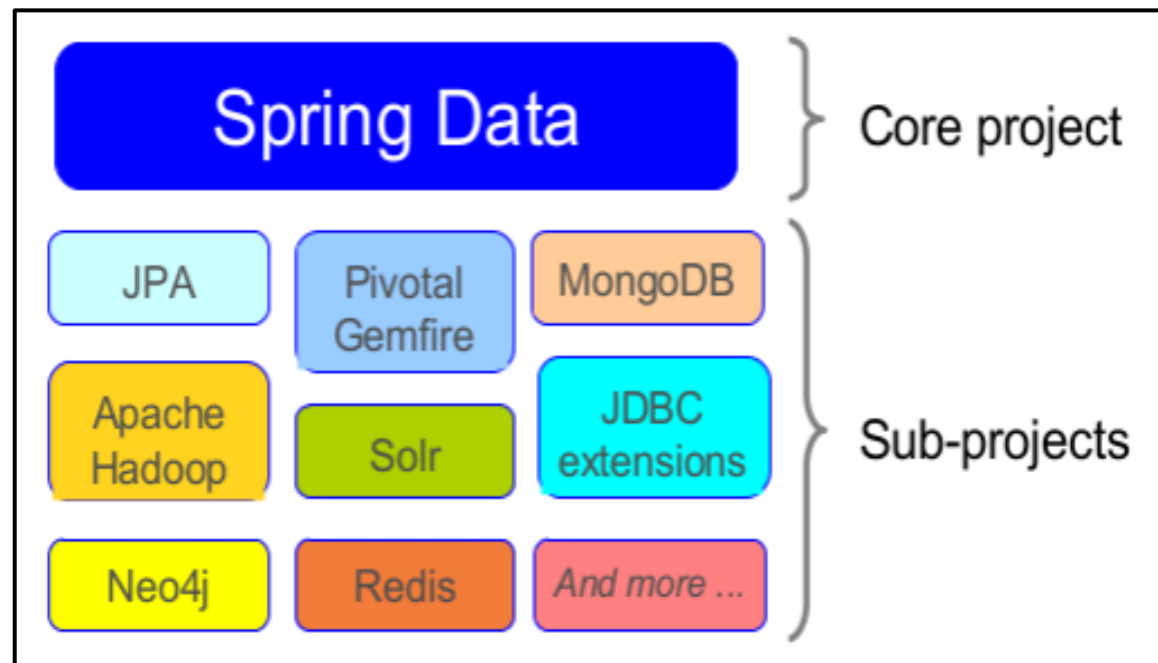
UP ASI
Bureau E204

Plan du Cours

- Spring Data
- Spring Data JPA
- CRUD Repository interface
- Créer et Utiliser un Repository
- TP Spring Boot + Spring Data JPA

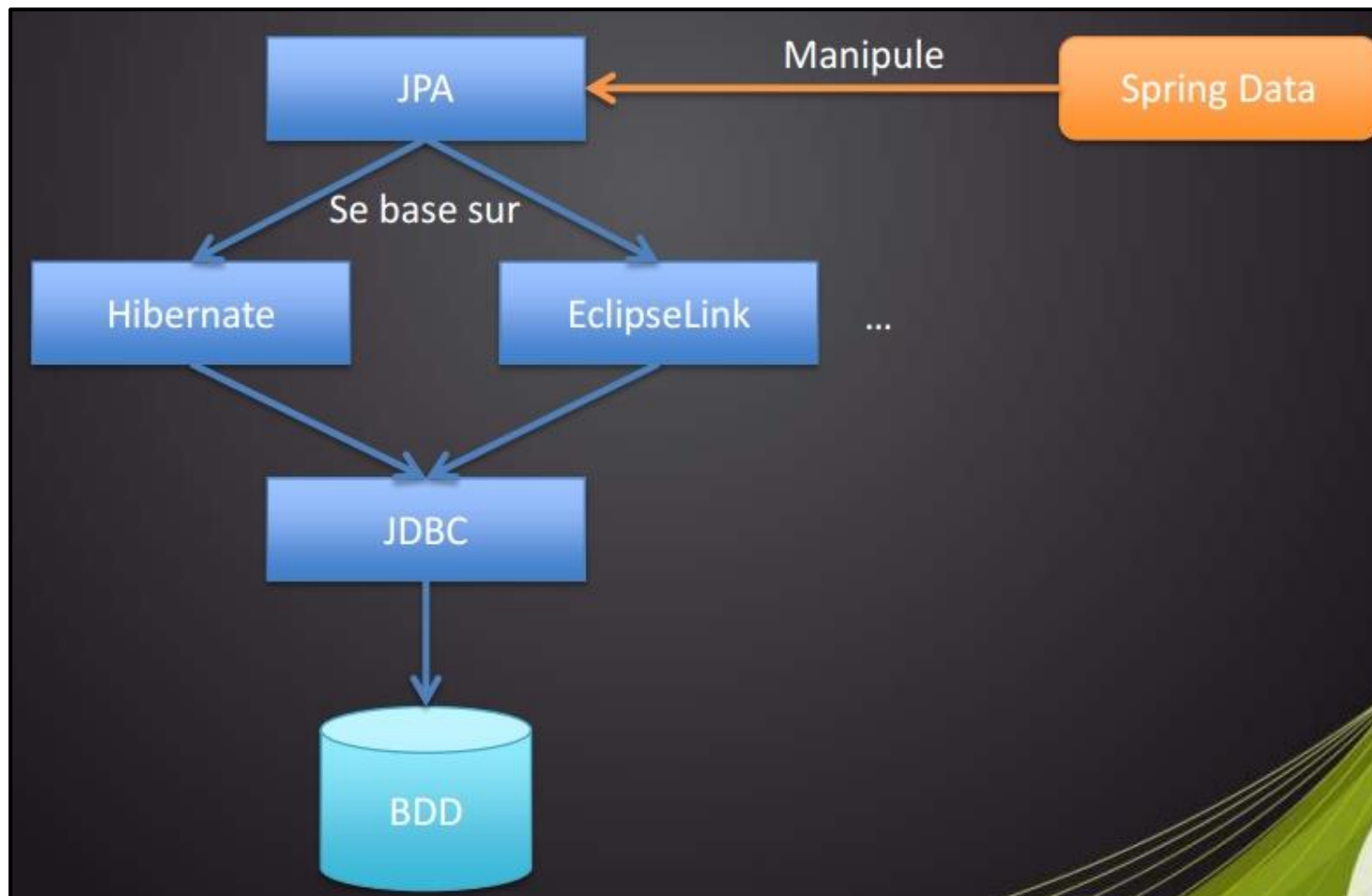
SPRING DATA

- C'est un module Spring qui a pour but de :
 - Faciliter l'écriture des couches d'accès aux données.
 - Offrir une abstraction commune pour l'accès aux données quelle que soit la source de données (SQL ou NoSQL).



SPRING DATA JPA

- Spring Data JPA est un sous projet du projet Spring Data.



SPRING DATA JPA

- Code répétitif avec les DAO (Data Access Object) avant Spring Data JPA:

```
public class ProjetDAO {  
  
    public Projet findById (Long id) {  
        //code ...  
    }  
  
    public List<Projet> findAll{  
        //code ...  
    }  
  
}
```

```
public class EntrepriseDAO{  
  
    public Entreprise findById (Long id) {  
        //code ...  
    }  
  
    public List<Entreprise> findAll{  
        //code ...  
    }  
  
}
```

SPRING DATA JPA

- Avec Spring Data JPA, il vous suffit de définir une interface qui permet de manipuler une entité, en étendant l'interface `CrudRepository <T, ID>` et de déclarer les méthodes pour manipuler cette entité.
- Spring Data JPA créera une classe qui implémente cette interface pour vous.
- Exemple :

```
public interface ProjetRepository extends JpaRepository<Projet, Long> {...}
```

- L'interface **ProjetRepository** étend l'interface **CrudRepository<Projet, Long>**. Elle contient les méthodes pour manipuler l'entité `Projet`.
- Spring Data JPA va automatiquement créer une classe qui implémente cette interface au moment de l'exécution de la l'application.

Keywords

- Spring Data JPA va créer le code pour les méthodes que tu souhaites utiliser. Il suffit de lui indiquer les méthodes à utiliser :

Keyword	Sample	Equivalent to
GreaterThan	<code>findByDateDebutGreaterThan(Date dateC);</code>	Select p from ProjetDetail p where p.dateDebut > :dateC
LessThan	<code>findByDateDebutLessThan(Date dateN);</code>	Select p from ProjetDetail p where p.dateDebut < :dateN
Between	<code>findByDateDebutBetween(Date dFrom, Date dTo);</code>	Select p from ProjetDetail p where p.dateDebut between :dFrom and :dTo
IsNull, NotNull	<code>findByCout_provisoireNotNull();</code>	Select p from ProjetDetail p where p.cout_provisoire is not null
IsNull, Null	<code>findByDescriptionNull();</code>	Select p from ProjetDetail p where p.description is null
Like	<code>findByTechnologieLike(String technologie);</code>	Select p from ProjetDetail p where p. technologie like :technologie
(No keyword)	<code>findByTechnologie(String technologie);</code>	Select p from ProjetDetail p where p. technologie= :technologie
(No keyword)	<code>findOne(ID primaryKey);</code>	Select p from ProjetDetail p where p.id =:primaryKey
(No keyword)	<code>Long count();</code>	Select count(*) From Projet;

Keywords

- Mots-clés des requêtes Spring Data JPA

Query subject keywords	find...By, read...By, get...By, search...By, stream...By, exists...By, count...By, delete...By, remove...By, ...Distinct...
Reserved methods	• deleteAllById(Iterable<ID> identifiers) • deleteById(ID identifier) • existsById(ID identifier) • findAllById(Iterable<ID> identifiers) • findById(ID identifier)

Keywords

- Mots-clés des requêtes Spring Data JPA

Query predicate Keywords	AND, OR, BEFORE, AFTER, BETWEEN, STARTING_WITH, CONTAINING, ENDING_WITH, EXISTS, FALSE, LESS_THAN GREATER_THAN, IN, IS, IS_EMPTY, IS_NOT_EMPTY, IS_NULL, IS_NOT_NULL, LIKE, NOT_LIKE
IgnoreCase, IgnoringCase, OrderBy	IgnoreCase, IgnoringCase, OrderBy

EXAMPLE

@Repository

```
public interface ProjetDetailRepository
extends JpaRepository<ProjetDetail, Long>
{
```

```
// SELECT * FROM ProjetDetail WHERE technologie LIKE '%in%';
```

```
List<ProjetDetail> findByTechnologieLike(String technologie);
```

```
List<ProjetDetail> findByTechnologieContains(String technologie);
```

```
List<ProjetDetail> findByTechnologieContaining(String technologie);}
```

```
@Entity
@Table(name = "T_PROJET_DETAIL")
public class Projet_Detail implements Serializable {
    private static final long serialVersionUID = 1L;

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "PD_ID")
    private Long id; // Identifiant projet detail (Clé p

    @Column(name = "PD_DESCRIPTION")
    private String description;

    @Column(name = "PD_TECHNOLOGIE")
    private String technologie;

    @Column(name = "PD_COUT_PROVISoire")
    private Long cout_provisoire;

    @Temporal(TemporalType.DATE)
    private Date dateDebut;
    @OneToOne(mappedBy="projetDetail")
    private Projet projet;
}
```

EXEMPLE

- Afficher la liste des projets qui ont une technologie précise.

@Repository

```
public interface ProjetRepository extends JpaRepository<Projet, Long> {  
    List<Projet> findByProjetDetailTechnologieContains(String technologie);  
}
```

association attribut dans la table Projet Detail

```
@Entity  
@Table(name = "T_PROJET")  
public class Projet implements Serializable {  
    private static final long serialVersionUID = 1L;  
  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    @Column(name = "PROJET_ID")  
    private Long id; // Identifiant projet (Clé primaire)  
  
    @Column(name = "PROJET_SUJET")  
    private String sujet;  
  
    @OneToOne  
    private Projet Detail projetDetail;  
}
```

EXEMPLE

- Afficher la liste des projets d'une équipe.

@Repository

```
public interface ProjetRepository {
```

```
    extends JpaRepository<Projet, Long> {
```

```
        List<Projet> findByEquipesIdEquipe(Long equipeId);
```

```
    }
```

```
@Entity
@Table(name = "T_PROJET")
public class Projet implements Serializable {
    private static final long serialVersionUID = 1L;

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "PROJET_ID")
    private Long id; // Identifiant projet (Clé primaire)

    @Column(name = "PROJET_SUJET")
    private String sujet;

    @ManyToMany(mappedBy = "projets", cascade = CascadeType.ALL)
    private Set<Equipe> equipes;
```

EXEMPLE

- Afficher la liste des projets d'une équipe dont la description est non nulle.

@Repository

```
public interface ProjetRepository extends JpaRepository<Projet, Long> {  
    List<Projet>  
    findByEquipesIdEquipeAndProjetDetailDescriptionNotNull(Long equipeId);  
}
```

EXEMPLE

@Repository

```
public interface ProjetRepository extends JpaRepository<Projet, Long> {
```

- Afficher la liste des projets par équipe et entreprise.

```
List<Projet> findByEquipesIdEquipeAndEquipesEntrepriseIdEntreprise  
(Long equipeId, Long entrepriseId);
```

- Afficher la liste des projets par la spécialité d'une équipe et l'adresse de l'entreprise.

```
List<Projet>  
findByEquipesSpecialiteContainsAndEquipesEntrepriseAdresseContains  
(String specialite, String adresse);  
}
```

EXEMPLE

- Afficher une entreprise selon son nom distinct.

Entreprise findDistinctByNom(String nom);

- Vérifier si une équipe existe avec le nom spécifié, sans tenir compte de la casse.

boolean existsByNomIgnoreCase(String nomEquipe);

- Afficher les équipes dont la spécialité contient « spring », triées par ordre décroissant du nom de leur entreprise.

```
List<Equipe> readBySpecialiteContainingOrderByEntrepriseNomDesc(String specialite);  
}
```

EXEMPLE

- Afficher les projets dont le nom commence par « Esp » et dont la description associé contient le mot « enseignement ».

```
List<Projet> findByNomStartingWithAndProjetDetailDescriptionContaining(String valeur1, String valeur2);
```

- Supprimer les projets dont la technologie figure dans la liste fournie.

```
void removeByProjetDetailTechnologieIn(List<String> listeTechnologies);
```

- Calculer le nombre de projets dont la date de début se situe entre deux dates données.

```
long countByProjetDetailDateDebutBetween(LocalDate date1, LocalDate date2);
```


SPRING DATA JPA (JPAREpository)

Si vous avez des questions, n'hésitez pas à nous contacter :

Département Informatique
UP Architectures des Systèmes d'Information
Bureau E204